AI ASSISTED CODING

LAB ASSIGNMENT-13.3

A.Srichala

2303A52435

CSE-AIML

BT-31

Lab 13: Code Refactoring – Improving Legacy Code with AI Suggestions
Lab Objectives:
• Identify code smells and inefficiencies in legacy Python scripts.
• Use AI-assisted coding tools to refactor for readability, maintainability, and performance.
• Apply modern Python best practices while ensuring output correctness.

Task Description #1 (Refactoring – Removing Code Duplication)
• Task: Use AI to refactor a given Python script that contains multiple repeated code blocks.
• Instructions:
o Prompt AI to identify duplicate logic and replace it with functions or classes.
o Ensure the refactored code maintains the same output.
o Add docstrings to all functions.
• Sample Legacy Code:

```
# Legacy script with repeated logic
print("Area of Rectangle:", 5 * 10)
print("Perimeter of Rectangle:", 2 * (5 + 10))
print("Area of Rectangle:", 7 * 12)
print("Perimeter of Rectangle:", 2 * (7 + 12))
print("Area of Rectangle:", 10 * 15)
print("Perimeter of Rectangle:", 2 * (10 + 15))
```

• Expected Output:
o Refactored code with a reusable function and no duplication.
o Well documented code

```python
print("Area of Rectangle:", 5 * 10)
print("Perimeter of Rectangle:", 2 * (5 + 10))
print("Area of Rectangle:", 7 * 12)
print("Perimeter of Rectangle:", 2 * (7 + 12))
print("Area of Rectangle:", 10 * 15)
print("Perimeter of Rectangle:", 2 * (10 + 15))
#refactor above code using function
def rectangle_area(length, width):
    return length * width
print("Area of Rectangle:", rectangle_area(5, 10))
print("Area of Rectangle:", rectangle_area(7, 12))
print("Area of Rectangle:", rectangle_area(10, 15))
```

Output:-

```
C:\Users\ankam\OneDrive\Desktop\AI
Area of Rectangle: 50
Perimeter of Rectangle: 30
Area of Rectangle: 84
Perimeter of Rectangle: 38
Area of Rectangle: 150
Perimeter of Rectangle: 50
Area of Rectangle: 50
Area of Rectangle: 84
Area of Rectangle: 150
```

Task Description #2 (Refactoring – Extracting Reusable Functions)

• Task: Use AI to refactor a legacy script where multiple calculations are embedded directly inside the main code block.

• Instructions:

o Identify repeated or related logic and extract it into reusable functions.

o Ensure the refactored code is modular, easy to read, and documented with docstrings.

• Sample Legacy Code:

```
# Legacy script with inline repeated logic
price = 250
tax = price * 0.18
total = price + tax
print("Total Price:", total)
price = 500
tax = price * 0.18
total = price + tax
print("Total Price:", total)
```

• Expected Output:

o Code with a function calculate_total(price) that can be reused

for multiple price inputs.
o Well documented code

```python
demo85.py > ...
def calculate_total(price):
    """
    Calculates the total price including 18% tax.
    Parameters:
    price (float): The original price of the item.
    Returns:
    float: Total price after adding tax.
    """
    tax = price * 0.18
    total = price + tax
    return total
# Using the reusable function
price1 = 250
price2 = 500
print("Total Price:", calculate_total(price1))
print("Total Price:", calculate_total(price2))
```

Output:-

```
C:\Users\ankam\OneDrive\Desktop\AIAC>C:/Users/ankam/App
Total Price: 295.0
Total Price: 590.0
```

Task Description #3: Refactoring Using Classes and Methods (Eliminating Redundant Conditional Logic)

Refactor a Python script that contains repeated if–elif–else grading logic by implementing a structured, object-oriented solution using a class and a method.

Problem Statement

The given script contains duplicated conditional statements used to assign grades based on student marks. This redundancy violates clean code principles and reduces maintainability.

You are required to refactor the script using a class-based design to improve modularity, reusability, and readability while preserving the original grading logic.

Mandatory Implementation Requirements

1. Class Name: GradeCalculator
2. Method Name: calculate_grade(self, marks)
3. The method must:

o Accept marks as a parameter.

o Return the corresponding grade as a string.

o The grading logic must strictly follow the conditions below:

- Marks ≥ 90 and ≤ 100 → "Grade A"
- Marks ≥ 80 → "Grade B"
- Marks ≥ 70 → "Grade C"
- Marks ≥ 40 → "Grade D"

▪ Marks ≥ 0 → "Fail"
Note: Assume marks are within the valid range of 0 to 100.
4. Include proper docstrings for:
o The class
o The method (with parameter and return descriptions)
5. The method must be reusable and called multiple times without rewriting conditional logic.
• Given code:

```
marks = 85
if marks >= 90:
print("Grade A")
elif marks >= 75:
print("Grade B")
else:
print("Grade C")

marks = 72
if marks >= 90:
print("Grade A")
elif marks >= 75:
print("Grade B")
else:
print("Grade C")
```

Expected Output:
• Define a class named GradeCalculator.
• Implement a method calculate_grade(self, marks) inside the class.

Create an object of the class.
• Call the method for different student marks.
• Print the returned grade values.

```python
marks = 85
if marks >= 90:
    print("Grade A")
elif marks >= 75:
    print("Grade B")
else:
    print("Grade C")
marks = 72
if marks >= 90:
    print("Grade A")
elif marks >= 75:
    print("Grade B")
else:
    print("Grade C")
# refactor above code and implement calculate_grade function and create object of the class and call the method for different marks and print the returned grade
class GradeCalculator:
    def calculate_grade(self, marks):
        if marks >= 90:
            return "Grade A"
        elif marks >= 75:
            return "Grade B"
        else:
            return "Grade C"
grade_calculator = GradeCalculator()
marks1 = 85
marks2 = 72
print("Marks:", marks1, "-", grade_calculator.calculate_grade(marks1))
print("Marks:", marks2, "-", grade_calculator.calculate_grade(marks2))
```

Output:-

```
C:\Users\ankam\OneDrive\Desktop\AIAC>C:/Users/ankam/AppData/Local/Programs/Python/Py
Grade B
Grade C
Marks: 85 - Grade B
Marks: 72 - Grade C
```

Task Description #4 (Refactoring – Converting Procedural Code to Functions)

• Task: Use AI to refactor procedural input–processing logic into functions.

Instructions:

o Identify input, processing, and output sections.

o Convert each into a separate function.

o Improve code readability without changing behavior.

• Sample Legacy Code:

num = int(input("Enter number: "))

square = num * num

print("Square:", square)

• Expected Output:

o Modular code using functions like get_input(), calculate_square(), and display_result()

```python
num = int(input("Enter number: "))
square = num * num
print("Square:", square)
#refactor the modular code using functions like get_input(), calculate_square(), and display_result()
def get_input():
    return int(input("Enter number: "))
def calculate_square(num):
    return num * num
def display_result(square):
    print("Square:", square)
number = get_input()
result = calculate_square(number)
display_result(result)
```

Output:-

```
C:\Users\ankam\OneDrive\Desktop\AIAC>C:/Users/ankam/AppData/Local/Programs/Pyth
Enter number: 5
Square: 25
Enter number: 6
Square: 36
```

Task 5 (Refactoring Procedural Code into OOP Design)

• Task: Use AI to refactor procedural code into a class-based design.

Focus Areas:

o Object-Oriented principles

o Encapsulation

Legacy Code:

salary = 50000

tax = salary * 0.2

net = salary - tax

print(net)

Expected Outcome:

o A class like EmployeeSalaryCalculator with methods and attributes.

```python
salary = 50000
tax = salary * 0.2
net = salary - tax
print(net)
#refactor the above code to use a class like EmployeeSalaryCalculator with methods and attributes.
class EmployeeSalaryCalculator:
    def __init__(self, salary):
        self.salary = salary

    def calculate_tax(self):
        return self.salary * 0.2

    def calculate_net_salary(self):
        tax = self.calculate_tax()
        return self.salary - tax
employee = EmployeeSalaryCalculator(50000)
net_salary = employee.calculate_net_salary()
print(net_salary)
```

Output:-

```
C:\Users\ankam\OneDrive\Desktop\AIAC>C:/Users/ankam/AppData/Local/Programs/Python/Pyth
40000.0
40000.0
```

Task 6 (Optimizing Search Logic)

• Task: Refactor inefficient linear searches using appropriate data structures.

• Focus Areas:

o Time complexity

o Data structure choice

Legacy Code:

users = ["admin", "guest", "editor", "viewer"]

name = input("Enter username: ")

found = False

for u in users:

if u == name:

found = True

print("Access Granted" if found else "Access Denied")

Expected Outcome:

o Use of sets or dictionaries with complexity justification

demo91.py > ...

```python
users = ["admin", "guest", "editor", "viewer"]
name = input("Enter username: ")
found = False
for u in users:
    if u == name:
        found = True
print("Access Granted" if found else "Access Denied")
#refactor the above code with the use of sets or dictionaries with complexity justification
users_set = set(users)  # Convert list to set for O(1) average time complexity lookups
if name in users_set:
    print("Access Granted")
else:
    print("Access Denied")
```

Output:

```
C:\Users\ankam\OneDrive\Desktop\AIAC>C:/Users/ankam/AppData/Local/Programs/Python/Pythor
Enter username: admin
Access Granted
Access Granted

C:\Users\ankam\OneDrive\Desktop\AIAC>C:/Users/ankam/AppData/Local/Programs/Python/Pythor
Enter username: abcdef
Access Denied
Access Denied
```

Task 7 – Refactoring the Library Management System

Problem Statement

You are provided with a poorly structured Library Management script that:

• Contains repeated conditional logic
• Does not use reusable functions
• Lacks documentation
• Uses print-based procedural execution
• Does not follow modular programming principles

Your task is to refactor the code into a proper format

1. Create a module library.py with functions:
o add_book(title, author, isbn)
o remove_book(isbn)
o search_book(isbn)
2. Insert triple quotes under each function and let Copilot complete the docstrings.

3. Generate documentation in the terminal.
4. Export the documentation in HTML format.
5. Open the file in a browser.

Given Code

```
# Library Management System (Unstructured Version)
# This code needs refactoring into a proper module with documentation.
library_db = {}
# Adding first book
title = "Python Basics"
author = "John Doe"
isbn = "101"
if isbn not in library_db:
library_db[isbn] = {"title": title, "author": author}
print("Book added successfully.")
else:
print("Book already exists.")
# Adding second book (duplicate logic)
title = "AI Fundamentals"
author = "Jane Smith"
isbn = "102"
if isbn not in library_db:
library_db[isbn] = {"title": title, "author": author}
print("Book added successfully.")
else:
print("Book already exists.")
# Searching book (repeated logic structure)
isbn = "101"

if isbn in library_db:
print("Book Found:", library_db[isbn])
else:
print("Book not found.")
# Removing book (again repeated pattern)
```

isbn = "101"
if isbn in library_db:
del library_db[isbn]
print("Book removed successfully.")
else:
print("Book not found.")
# Searching again
isbn = "101"
if isbn in library_db:
print("Book Found:", library_db[isbn])
else:
print("Book not found.")

demo93.py > ...

```python
# Library Management System (Unstructured Version)
# This code needs refactoring into a proper module with documentation.
library_db = {}
# Adding first book
title = "Python Basics"
author = "John Doe"
isbn = "101"

if isbn not in library_db:
    library_db[isbn] = {"title": title, "author": author}
    print("Book added successfully.")
else:
    print("Book already exists.")

# Adding second book (duplicate logic)
title = "AI Fundamentals"
author = "Jane Smith"
isbn = "102"

if isbn not in library_db:
    library_db[isbn] = {"title": title, "author": author}
    print("Book added successfully.")
else:
    print("Book already exists.")

# Searching book (repeated logic structure)
isbn = "101"
if isbn in library_db:
    print("Book Found:", library_db[isbn])
else:
    print("Book not found.")

# Removing book (again repeated pattern)
isbn = "101"
if isbn in library_db:
    del library_db[isbn]
    print("Book removed successfully.")
else:
    print("Book not found.")

# Searching again
```

```python
# Searching again
isbn = "101"
if isbn in library_db:
    print("Book Found:", library_db[isbn])
else:
    print("Book not found.")
# Refactor the above Library Management System code into a module named library.py.
# Create reusable functions:
# add_book(title, author, isbn)
# remove_book(isbn)
# search_book(isbn)
# Add triple quotes ("""") under each function so Copilot generates docstrings.
# Remove duplicate logic and follow modular programming.
# Generate documentation using:
# python -m pydoc library
# Export HTML documentation using:
# python -m pydoc -w library
# Open library.html in a browser.
# Provide the complete refactored library.py code.
# library.py
library_db = {}
def add_book(title, author, isbn):
    """Add a book to the library database."""
    if isbn not in library_db:
        library_db[isbn] = {"title": title, "author": author}
        print("Book added successfully.")
    else:
        print("Book already exists.")
def remove_book(isbn):
    """Remove a book from the library database."""
    if isbn in library_db:
        del library_db[isbn]
        print("Book removed successfully.")
    else:
        print("Book not found.")
def search_book(isbn):
    """Search for a book in the library database."""
    if isbn in library_db:
        print("Book Found:", library_db[isbn])
    else:
        print("Book not found.")
```

Output:

```
C:\Users\ankam\OneDrive\Desktop\AIAC>C:/Users/ankam/AppData/Local/Programs/Python/Python312/pyt
Book added successfully.
Book added successfully.
Book Found: {'title': 'Python Basics', 'author': 'John Doe'}
Book removed successfully.
Book not found.
```

Task 8– Fibonacci Generator

Write a program to generate Fibonacci series up to n.

The initial code has:

• Global variables.

• Inefficient loop.

• No functions or modularity.

Task for Students:

• Refactor into a clean reusable function (generate_fibonacci).

• Add docstrings and test cases.

• Compare AI-refactored vs original.

Bad Code Version:

```
# fibonacci bad version
n=int(input("Enter limit: "))
a=0

b=1
print(a)
print(b)
for i in range(2,n):
c=a+b
print(c)
a=b
b=c
```

demo94.py > ...

```python
# fibonacci bad version
n=int(input("Enter limit: "))
a=0
b=1
print(a)
print(b)
for i in range(2,n):
 c=a+b
 print(c)
 a=b
 b=c
#refractor the above code into a clean reusabble function
# add docstrings and testcases
# compare AI refractored code with the original code
def fibonacci(n):
    """
    Generate Fibonacci sequence up to n terms.

    Parameters:
    n (int): The number of terms to generate.

    Returns:
    list: A list containing the Fibonacci sequence up to n terms.
    """
    if n <= 0:
        return []
    elif n == 1:
        return [0]
    elif n == 2:
        return [0, 1]

    sequence = [0, 1]
    for i in range(2, n):
        c = sequence[i-1] + sequence[i-2]
        sequence.append(c)

    return sequence
# Test cases
print(fibonacci(10))  # Expected output: [0, 1, 1, 2, 3, 5, 8, 13, 21, 34]
print(fibonacci(0))   # Expected output: []
print(fibonacci(1))   # Expected output: [0]
print(fibonacci(2))   # Expected output: [0, 1]
print(fibonacci(3))   # Expected output: [0, 1, 1]
print(fibonacci(4))   # Expected output: [0, 1, 1, 2]
print(fibonacci(5))   # Expected output: [0, 1, 1, 2, 3]
print(fibonacci(6))   # Expected output: [0, 1, 1, 2, 3, 5]
print(fibonacci(7))   # Expected output: [0, 1, 1, 2, 3, 5, 8]
print(fibonacci(8))   # Expected output: [0, 1, 1, 2, 3, 5, 8, 13]
print(fibonacci(9))   # Expected output: [0, 1, 1, 2, 3, 5, 8, 13, 21]
print(fibonacci(11))  # Expected output: [0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55]
```

Output:-

```
C:\Users\ankam\OneDrive\Desktop\AIAC>C:/Users/ankam/AppData/Local/Programs/Python
Enter limit: 3
0
1
1
[0, 1, 1, 2, 3, 5, 8, 13, 21, 34]
[]
[0]
[0, 1]
[0, 1, 1]
[0, 1, 1, 2]
[0, 1, 1, 2, 3]
[0, 1, 1, 2, 3, 5]
[0, 1, 1, 2, 3, 5, 8]
[0, 1, 1, 2, 3, 5, 8, 13]
[0, 1, 1, 2, 3, 5, 8, 13, 21]
[0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55]
```

Task 9 – Twin Primes Checker

Twin primes are pairs of primes that differ by 2 (e.g., 11 and 13, 17 and 19).

The initial code has:

• Inefficient prime checking.

• No functions.

• Hardcoded inputs.

Task for Students:

• Refactor into is_prime(n) and is_twin_prime(p1, p2).

• Add docstrings and optimize.

• Generate a list of twin primes in a given range using AI.

Bad Code Version:

```
# twin primes bad version
a=11
b=13
fa=0
for i in range(2,a):
if a%i==0:
fa=1
fb=0
for i in range(2,b):
if b%i==0:
fb=1
if fa==0 and fb==0 and abs(a-b)==2:
print("Twin Primes")
else:
print("Not Twin Primes")
```

```python
# twin primes bad version
a = 11
b = 13

fa = 0
for i in range(2, a):
    if a % i == 0:
        fa = 1

fb = 0
for i in range(2, b):
    if b % i == 0:
        fb = 1

if fa == 0 and fb == 0 and abs(a - b) == 2:
    print("Twin Primes")
else:
    print("Not Twin Primes")

#Refractor the above code into is_prime(n) and is_twin_prime(p1, p2).
# Add docstrings and optimize.
# Generate a list of twin primes in a given range using AI.

def is_prime(n):
    """
    Check if a number is prime.

    Parameters:
    n (int): The number to check.

    Returns:
    bool: True if the number is prime, False otherwise.
    """
    if n <= 1:
        return False
    if n <= 3:
        return True
    if n % 2 == 0 or n % 3 == 0:
        return False
    i = 5
    while i * i <= n:
        if n % i == 0 or n % (i + 2) == 0:
            return False
        i += 6
    return True
```

```python

def is_twin_prime(p1, p2):
    """
    Check if two numbers are twin primes.

    Parameters:
    p1 (int): The first number.
    p2 (int): The second number.

    Returns:
    bool: True if the numbers are twin primes, False otherwise.
    """
    return is_prime(p1) and is_prime(p2) and abs(p1 - p2) == 2

def generate_twin_primes(start, end):
    """
    Generate a list of twin primes in a given range.

    Parameters:
    start (int): The starting number of the range.
    end (int): The ending number of the range.

    Returns:
    list: A list of tuples containing twin primes in the given range.
    """
    twin_primes = []
    for i in range(start, end - 1):
        if is_twin_prime(i, i + 2):
            twin_primes.append((i, i + 2))
    return twin_primes

# Test cases
print(generate_twin_primes(1, 30))  # Expected output: [(3, 5), (5, 7), (11, 13), (17, 19), (29, 31)]

```

Output:-

```
C:\Users\ankam\OneDrive\Desktop\AIAC>C:/Users/ankam/AppData/Local/Programs/Python/Python3
Twin Primes
[(3, 5), (5, 7), (11, 13), (17, 19)]
```

Task 10 – Refactoring the Chinese Zodiac Program

Objective

Refactor the given poorly structured Python script into a clean, modular, and reusable implementation.

The current program reads a year from the user and prints the corresponding Chinese Zodiac sign. However, the implementation contains repetitive

conditional logic, lacks modular design, and does not follow clean coding principles.

Your task is to refactor the code to improve readability, maintainability, and structure.

Chinese Zodiac Cycle (Repeats Every 12 Years)

1. Rat
2. Ox
3. Tiger
4. Rabbit
5. Dragon
6. Snake

7. Horse
8. Goat (Sheep)
9. Monkey
10. Rooster
11. Dog
12. Pig

```
# Chinese Zodiac Program (Unstructured Version)
# This code needs refactoring.
year = int(input("Enter a year: "))
if year % 12 == 0:
print("Monkey")
elif year % 12 == 1:
print("Rooster")
elif year % 12 == 2:
print("Dog")
elif year % 12 == 3:
print("Pig")
elif year % 12 == 4:
print("Rat")
elif year % 12 == 5:
print("Ox")
elif year % 12 == 6:
print("Tiger")
elif year % 12 == 7:
print("Rabbit")
elif year % 12 == 8:
print("Dragon")

elif year % 12 == 9:
print("Snake")
elif year % 12 == 10:
print("Horse")
elif year % 12 == 11:
print("Goat")
```

You must:
1. Create a reusable function: get_zodiac(year)
2. Replace the if-elif chain with a cleaner structure (e.g., list or dictionary).
3. Add proper docstrings.
4. Separate input handling from logic.
5. Improve readability and maintainability.
6. Ensure output remains correct.

demo96.py > ...

```python
# Chinese Zodiac Program (Unstructured Version)

year = int(input("Enter a year: "))

if year % 12 == 0:
    print("Monkey")
elif year % 12 == 1:
    print("Rooster")
elif year % 12 == 2:
    print("Dog")
elif year % 12 == 3:
    print("Pig")
elif year % 12 == 4:
    print("Rat")
elif year % 12 == 5:
    print("Ox")
elif year % 12 == 6:
    print("Tiger")
elif year % 12 == 7:
    print("Rabbit")
elif year % 12 == 8:
    print("Dragon")
elif year % 12 == 9:
    print("Snake")
elif year % 12 == 10:
    print("Horse")
elif year % 12 == 11:
    print("Goat")

# Refactor the above code into Create a reusable function: get_zodiac(year)
# Replace the if-elif chain with a cleaner structure (e.g., list or dictionary).
# Add proper docstrings.
# Separate input handling from logic.
# Improve readability and maintainability.
# Ensure output remains correct
def get_zodiac(year):
    """
    Returns the Chinese Zodiac animal for a given year.

    Parameters:
        year (int): The year provided by the user.

    Returns:
        str: The corresponding Chinese Zodiac animal.
    """
```

```python

    Returns:
        str: The corresponding Chinese Zodiac animal.
    """

    zodiac_cycle = [
        "Monkey", "Rooster", "Dog", "Pig",
        "Rat", "Ox", "Tiger", "Rabbit",
        "Dragon", "Snake", "Horse", "Goat"
    ]

    return zodiac_cycle[year % 12]


def main():
    """Handle user input and display the zodiac result."""
    year = int(input("Enter a year: "))
    animal = get_zodiac(year)
    print("Chinese Zodiac:", animal)


if __name__ == "__main__":
    main()
```

Output:

```
C:\Users\ankam\OneDrive\Desktop\AIAC>C:/Users/ankam/AppData/Local/Programs/Python/Python312/python.ex
Enter a year: 2026
Horse
Enter a year: 2027
Chinese Zodiac: Goat
```

Task 11 – Refactoring the Harshad (Niven) Number Checker

Refactor the given poorly structured Python script into a clean, modular, and reusable implementation.

A Harshad (Niven) number is a number that is divisible by the sum of its digits.

For example:

- 18 → 1 + 8 = 9 → 18 ÷ 9 = 2 (Harshad Number)
- 19 → 1 + 9 = 10 → 19 ÷ 10 ≠ integer (Not Harshad)

Problem Statement

The current implementation:

- Mixes logic and input handling
- Uses redundant variables
- Does not use reusable functions properly
- Returns print statements instead of boolean values
- Lacks documentation

You must refactor the code to follow clean coding principles.

```
# Harshad Number Checker (Unstructured Version)
num = int(input("Enter a number: "))
temp = num
```

sum_digits = 0
while temp > 0:

digit = temp % 10
sum_digits = sum_digits + digit
temp = temp // 10
if sum_digits != 0:
if num % sum_digits == 0:
print("True")
else:
print("False")
else:
print("False")
You must:
1. Create a reusable function: is_harshad(number)
2. The function must:
o Accept an integer parameter.
o Return True if the number is divisible by the sum of its digits.
o Return False otherwise.
3. Separate user input from core logic.
4. Add proper docstrings.
5. Improve readability and maintainability.
6. Ensure the program handles edge cases (e.g., 0, negative numbers).

```python
# Harshad Number Checker (Unstructured Version)

num = int(input("Enter a number: "))
temp = num
sum_digits = 0

while temp > 0:
    digit = temp % 10
    sum_digits = sum_digits + digit
    temp = temp // 10

if sum_digits != 0:
    if num % sum_digits == 0:
        print("True")
    else:
        print("False")
else:
    print("False")
#refractor the above code and Create a reusable function: is_harshad(number)
#The function must:
#1. Accept an integer parameter.
#2. Return True if the number is divisible by the sum of its digits.
#3. Return False otherwise.
#4. Separate user input from core logic.
#5. Add proper docstrings.
#6. Improve readability and maintainability.
#7. Ensure the program handles edge cases (e.g., 0, negative numbers).
def is_harshad(number):
    """
    Check if a number is a Harshad number.

    A Harshad number is an integer that is divisible by the sum of its digits.
```

```python

    A Harshad number is an integer that is divisible by the sum of its digits.

    Parameters:
    number (int): The integer to check.

    Returns:
    bool: True if the number is a Harshad number, False otherwise.
    """
    if number < 0:
        return False  # Harshad numbers are typically defined for non-negative integers

    sum_digits = sum(int(digit) for digit in str(number))

    if sum_digits == 0:
        return False  # Avoid division by zero

    return number % sum_digits == 0
def main():
    """Main function to execute the Harshad number checker.
    """
    try:
        num = int(input("Enter a number: "))
        if is_harshad(num):
            print(f"{num} is a Harshad number.")
        else:
            print(f"{num} is not a Harshad number.")
    except ValueError:
        print("Invalid input. Please enter an integer.")
if __name__ == "__main__":    main()

```

Output:

```
C:\Users\ankam\OneDrive\Desktop\AIAC>C:/Users/ankam/AppData/Local/Programs/Python/Python312/python.ex
Enter a number: 3
True
Enter a number: 3
3 is a Harshad number.
```

Task 12 – Refactoring the Factorial Trailing Zeros Program

Refactor the given poorly structured Python script into a clean, modular, and efficient implementation.

The program calculates the number of trailing zeros in n! (factorial of n).

Problem Statement

The current implementation:

- Calculates the full factorial (inefficient for large n)
- Mixes input handling with business logic
- Uses print statements instead of return values
- Lacks modular structure and documentation

You must refactor the code to improve efficiency, readability, and maintainability.

```
# Factorial Trailing Zeros (Unstructured Version)
n = int(input("Enter a number: "))
fact = 1
i = 1
while i <= n:
fact = fact * i
```

i = i + 1
count = 0
while fact % 10 == 0:
count = count + 1
fact = fact // 10
print("Trailing zeros:", count)
You must:
1. Create a reusable function: count_trailing_zeros(n)
2. The function must:
o Accept a non-negative integer n.
o Return the number of trailing zeros in n!.
3. Do NOT compute the full factorial.
4. Use an optimized mathematical approach (count multiples of 5).
5. Add proper docstrings.
6. Separate user interaction from core logic.
7. Handle edge cases (e.g., negative numbers, zero).

demo98.py > ...

```python
# Factorial Trailing Zeros (Unstructured Version)

from ast import Add


n = int(input("Enter a number: "))

fact = 1
i = 1

while i <= n:
    fact = fact * i
    i = i + 1

count = 0

while fact % 10 == 0:
    count = count + 1
    fact = fact // 10

print("Trailing zeros:", count)
#refactor the above code and create a reusable function: count_trailing_zeros(n)
#2. The function must:
#o Accept a non-negative integer n.
#o Return the number of trailing zeros in n!.
#3. Do NOT compute the full factorial.
#4. Use an optimized mathematical approach (count multiples of 5).
#5. Add proper docstrings.
#6. Separate user interaction from core logic.
#7. Handle edge cases (e.g., negative numbers, zero).
def count_trailing_zeros(n):
    """Count the number of trailing zeros in n! (factorial of n).

    This function calculates the number of trailing zeros in the factorial of a non-negative integer n
    by counting the number of times 5 is a factor in the numbers from 1 to n.
```

```python
#7. Handle edge cases (e.g., negative numbers, zero).
def count_trailing_zeros(n):
    """Count the number of trailing zeros in n! (factorial of n).

    This function calculates the number of trailing zeros in the factorial of a non-negative integer n
    by counting the number of times 5 is a factor in the numbers from 1 to n.

    Args:
        n (int): A non-negative integer for which to count trailing zeros in its factorial.

    Returns:
        int: The number of trailing zeros in n!.
    """
    if n < 0:
        raise ValueError("Input must be a non-negative integer.")
    count = 0
    power_of_5 = 5
    while n >= power_of_5:
        count += n // power_of_5
        power_of_5 *= 5
    return count
def main():
    """Main function to execute the trailing zeros counter."""
    try:
        num = int(input("Enter a non-negative integer: "))
        zeros = count_trailing_zeros(num)
        print(f"The number of trailing zeros in {num}! is: {zeros}")
    except ValueError as e:
        print(e)
if __name__ == "__main__":    main()
```

Output:-

```
C:\Users\ankam\OneDrive\Desktop\AIAC>C:/Users/ankam/AppData/Local/Programs/Py
Enter a number: 1
Trailing zeros: 0
Enter a non-negative integer: 7
The number of trailing zeros in 7! is: 1
```

Task Description #13: Smart Airport Management System – Data Structure Challenge

An airport authority plans to implement:

1. Passenger Check-In Queue – Process passengers in order of arrival.
2. Emergency Landing Priority – Aircraft in emergency get priority.
3. Flight Information Lookup – Fast search using Flight ID.
4. Air Route Connectivity – Airports connected via routes.
5. Boarding Process (Last-In First-Out Cabin Storage) – Overhead luggage management simulation.

Student Task:

- Choose suitable data structures from the provided list.
- Provide 2–3 sentence justification per feature.
- Implement one feature in Python with comments and docstrings using AI assistance.

Expected Output:

- Mapping table.
- Working Python implementation.

Feature	Data Structure	Justification
Passenger Check-In Queue	Queue	A queue follows **FIFO (First-In-First-Out)** order. Passengers who arrive first should be processed first during check-in. This ensures fairness and orderly processing.
Emergency Landing Priority	Priority Queue	A priority queue processes elements based on **priority instead of arrival time**. Aircraft with emergency situations are given higher priority and allowed to land before others.
Flight Information Lookup	Hash Table	A hash table allows **fast searching using a key**. Using Flight ID as the key enables very quick lookup of flight details.
Air Route Connectivity	Graph	Airports and routes naturally form a **network structure**. Airports can be represented as nodes and flight routes as edges in a graph.
Boarding Process (Cabin Storage)	Stack	A stack follows **LIFO (Last-In-First-Out)**. The last luggage placed in the overhead cabin is usually removed first when passengers retrieve bags.

Prompt:-

demo100.py > ...

```python
from collections import deque, defaultdict
import heapq

"""
Smart Airport Management System - Data Structure Design
Demonstrates optimal data structures for airport operations.
"""


# MAPPING TABLE: Feature → Data Structure
FEATURE_TO_DS = {
    "Passenger Check-In Queue": "Queue (deque)",
    "Emergency Landing Priority": "Min-Heap (heapq)",
    "Flight Information Lookup": "Hash Map (dict)",
    "Air Route Connectivity": "Graph (adjacency list)",
    "Boarding Process (Cabin Storage)": "Stack (list)"
}

print("=" * 60)
print("FEATURE → DATA STRUCTURE MAPPING")
print("=" * 60)

justifications = {
    "Passenger Check-In Queue":
        "Uses FIFO principle where first passenger to arrive checks in first. "
        "Queue (deque) ensures O(1) enqueue/dequeue operations.",

    "Emergency Landing Priority":
        "Requires processing flights by fuel criticality and time to landing. "
        "Min-Heap enables O(log n) insertion and O(1) retrieval of highest-priority flights.",

```

```python

    "Emergency Landing Priority":
        "Requires processing flights by fuel criticality and time to landing. "
        "Min-Heap enables O(log n) insertion and O(1) retrieval of highest-priority flights.",

    "Flight Information Lookup":
        "Needs instant access to flight details by flight number. "
        "Hash Map provides O(1) average-case lookup time.",

    "Air Route Connectivity":
        "Represents connections between airports for route planning. "
        "Adjacency list efficiently represents sparse graphs with O(V+E) traversal.",

    "Boarding Process (Cabin Storage)":
        "Last item loaded (overhead bin closure) must be first unloaded. "
        "Stack (LIFO) ensures O(1) push/pop operations."
}

for feature, ds in FEATURE_TO_DS.items():
    print(f"\n{feature}")
    print(f"  Data Structure: {ds}")
    print(f"  Justification: {justifications[feature]}")

# IMPLEMENTATION: Passenger Check-In Queue
print("\n" + "=" * 60)
print("IMPLEMENTATION: Passenger Check-In Queue System")
print("=" * 60)

class CheckInQueue:
    """
    Manages passenger check-in using a FIFO queue.
```

demo100.py > ...

```python
class CheckInQueue:
    """
    Manages passenger check-in using a FIFO queue.
    Tracks passenger data and processes check-ins sequentially.
    """

    def __init__(self):
        """Initialize an empty check-in queue."""
        self.queue = deque()
        self.ticket_counter = 1000

    def add_passenger(self, name, flight):
        """
        Add a passenger to the check-in queue.

        Args:
            name (str): Passenger name
            flight (str): Flight code (e.g., 'BA123')
        """
        ticket = self.ticket_counter
        self.ticket_counter += 1
        self.queue.append({"ticket": ticket, "name": name, "flight": flight})
        print(f"✓ {name} added to queue (Ticket: {ticket})")

    def check_in(self):
        """
        Process the next passenger in check-in queue.

        Returns:
            dict: Checked-in passenger data, or None if queue empty
        """
```

```python
        Returns:
            dict: Checked-in passenger data, or None if queue empty
        """
        if not self.queue:
            print("⚠ Queue is empty!")
            return None
        passenger = self.queue.popleft()
        print(f"✓ {passenger['name']} checked in (Flight: {passenger['flight']})")
        return passenger

    def queue_length(self):
        """Return the number of passengers waiting."""
        return len(self.queue)


# Demo
print("\nDemo:")
check_in = CheckInQueue()
check_in.add_passenger("Alice Johnson", "BA123")
check_in.add_passenger("Bob Smith", "AA456")
check_in.add_passenger("Carol White", "BA123")
print(f"Passengers in queue: {check_in.queue_length()}")
check_in.check_in()
check_in.check_in()
print(f"Remaining passengers: {check_in.queue_length()}")
```

Output:-

```
PROBLEMS   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS                                   Python ⚠ + ···

C:\Users\ankam\OneDrive\Desktop\AIAC>C:/Users/ankam/AppData/Local/Programs/Python/Python312/python.exe c:/Users/ankam/OneDrive/Desktop/AIAC/demo100.py
============================================================
FEATURE → DATA STRUCTURE MAPPING
============================================================

Passenger Check-In Queue
  Data Structure: Queue (deque)
  Justification: Uses FIFO principle where first passenger to arrive checks in first. Queue (deque) ensures O(1) enqueue/dequeue operations.

Emergency Landing Priority
  Data Structure: Min-Heap (heapq)
  Justification: Requires processing flights by fuel criticality and time to landing. Min-Heap enables O(log n) insertion and O(1) retrieval of highes
t-priority flights.

Flight Information Lookup
  Data Structure: Hash Map (dict)
  Justification: Needs instant access to flight details by flight number. Hash Map provides O(1) average-case lookup time.

Air Route Connectivity
  Data Structure: Graph (adjacency list)
  Justification: Represents connections between airports for route planning. Adjacency list efficiently represents sparse graphs with O(V+E) traversal
.

Boarding Process (Cabin Storage)
  Data Structure: Stack (list)
  Justification: Last item loaded (overhead bin closure) must be first unloaded. Stack (LIFO) ensures O(1) push/pop operations.

============================================================
IMPLEMENTATION: Passenger Check-In Queue System
============================================================

Demo:
✓ Alice Johnson added to queue (Ticket: 1000)
✓ Bob Smith added to queue (Ticket: 1001)
✓ Carol White added to queue (Ticket: 1002)
Passengers in queue: 3
✓ Alice Johnson checked in (Flight: BA123)
✓ Bob Smith checked in (Flight: AA456)
Remaining passengers: 1

C:\Users\ankam\OneDrive\Desktop\AIAC>
Ln 107, Col 58   Spaces: 4   UTF-8   {} Python   3.12.0   Python 3.12 (64-bi
```

Task Description #14: Smart Social Media Platform – Data Structure Selection
A social media company wants to develop a system that includes:
1. News Feed Generation – Display posts chronologically.
2. Trending Hashtags Tracker – Rank hashtags by usage frequency.
3. User Profile Search – Quick lookup using username.
4. Friend Network Representation – Represent user connections.
5. Message Undo Feature – Allow last sent message to be withdrawn.
Student Task:
• Select appropriate data structures from the list.
• Justify your choices clearly.
• Implement one selected feature as a Python program using AI-assisted code generation.
Expected Output:
• Table mapping features to data structures with justification.
• Working Python code with docstrings and comments.

Prompt:-

Smart Social Media Platform – Data Structure Selection

A social media system has these features:

1. News Feed Generation – Display posts chronologically

2. Trending Hashtags Tracker – Rank hashtags by usage frequency

3. User Profile Search – Quick lookup using username

4. Friend Network Representation – Represent user connections

5. Message Undo Feature – Undo the last sent message

Tasks:

1. Choose the most suitable data structure for each feature.

2. Write 2–3 sentences explaining why that data structure is suitable.

3. Implement one feature in Python with comments.

1. News Feed Generation → Deque / Queue

A Queue or Deque is suitable because posts are displayed in the order they are created. New posts are added to the end and older posts stay at the front. This maintains chronological order for the news feed.

2. Trending Hashtags Tracker → Priority Queue / Heap

A Priority Queue helps rank hashtags based on usage frequency. Hashtags with higher counts get higher priority. This allows the system to quickly show the most trending hashtags.

3. User Profile Search → Hash Table (Dictionary)

A Hash Table allows very fast lookup using a username as the key. The system can instantly retrieve a user profile without searching the entire list. This makes profile search efficient.

4. Friend Network Representation → Graph

A Graph is ideal because users can be represented as nodes and friendships as edges. It helps represent complex social connections. Graphs also allow operations like mutual friends or connection suggestions.

5. Message Undo Feature → Stack

A Stack follows the Last-In-First-Out principle. The last sent message is placed on top of the stack. When the user presses undo, the most recent message is removed first.

demo101.py > ...

```python
from collections import deque
from datetime import datetime

class NewsPost:
    def __init__(self, user_id, content, timestamp=None):
        self.user_id = user_id
        self.content = content
        self.timestamp = timestamp or datetime.now()

class NewsFeed:
    def __init__(self):
        # Deque maintains chronological order efficiently
        self.feed = deque()

    def add_post(self, post):
        """Add a new post to the end of the feed"""
        self.feed.append(post)

    def display_feed(self, limit=10):
        """Display posts in reverse chronological order (newest first)"""
        displayed = 0
        for post in reversed(self.feed):
            if displayed >= limit:
                break
            print(f"{post.timestamp} - User {post.user_id}: {post.content}")
            displayed += 1

    def remove_oldest_post(self):
        """Remove the oldest post (for memory management)"""
        if self.feed:
            self.feed.popleft()

# Example usage
news_feed = NewsFeed()
news_feed.add_post(NewsPost(101, "Just finished coding!"))
news_feed.add_post(NewsPost(102, "Hello world!"))
news_feed.add_post(NewsPost(103, "Learning Python"))
news_feed.display_feed()
```

Output:-

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

C:\Users\ankam\OneDrive\Desktop\AIAC>C:/Users/ankam/AppData/Local/Programs/Python/Python312
2026-03-11 22:06:19.059796 - User 103: Learning Python
2026-03-11 22:06:19.059796 - User 102: Hello world!
2026-03-11 22:06:19.059796 - User 101: Just finished coding!
```

Task #15: Smart University Hostel Management

Scenario:

A university wants to build a hostel management system that manages:

1. Room Allotment
2. Medical Emergency Handling
3. Student Record Retrieval
4. Hostel Block Connectivity
5. Visitor Entry Log

Student Task

• For each feature, select the most appropriate data structure from the list

below:

o Stack
o Queue
o Priority Queue
o Linked List
o Binary Search Tree (BST)
o Graph
o Hash Table
o Deque

• Justify your choice in 2–3 sentences per feature.
• Implement one selected feature as a working Python program with AI-assisted code generation.

Expected Output:

• A table mapping feature → chosen data structure → justification.
• A functional Python program implementing the chosen feature with comments and docstrings.

Feature	Data Structure	Justification
Room Allotment	Queue	A Queue works on the First-In-First-Out principle. Students who apply earlier for rooms should get rooms first. This ensures fair allocation of hostel rooms.
Medical Emergency Handling	Priority Queue	In medical emergencies, students with more serious conditions must be treated first. A Priority Queue allows higher priority cases to be processed before others. This ensures faster response for critical situations.
Student Record Retrieval	Hash Table	A Hash Table allows fast access to student records using a unique student ID. It provides constant-time lookup. This makes retrieving student information very efficient.
Hostel Block Connectivity	Graph	A Graph represents relationships between different hostel blocks. Each block can be a node and paths between them are edges. This helps in navigation and route planning within the hostel campus.
Visitor Entry Log	Stack	A Stack follows Last-In-First-Out order. The most recent visitor entry can be checked or removed first. This is useful for tracking the latest visitors entering the hostel.

“”“

Smart University Hostel Management – Data Structure Selection

A university hostel system must manage:

1. Room Allotment

2. Medical Emergency Handling

3. Student Record Retrieval

4. Hostel Block Connectivity

5. Visitor Entry Log

Tasks:

1. Choose the most appropriate data structure from the following list:

   Stack, Queue, Priority Queue, Linked List, Binary Search Tree (BST), Graph, Hash Table, Deque.

2. Justify your choice in 2–3 sentences for each feature.

3. Implement one selected feature as a Python program using comments and docstrings.

“”“

```python
"""
Smart University Hostel Management System
Data Structure Implementation for Medical Emergency Handling
"""

from collections import deque
from enum import Enum

class PriorityLevel(Enum):
    """Priority levels for medical emergencies"""
    CRITICAL = 1
    HIGH = 2
    MEDIUM = 3
    LOW = 4

class EmergencyRequest:
    """Represents a medical emergency request"""
    def __init__(self, student_id, name, priority, description):
        """
        Initialize emergency request

        Args:
            student_id: Unique identifier for student
            name: Student name
            priority: PriorityLevel enum value
            description: Emergency description
        """
        self.student_id = student_id
        self.name = name
        self.priority = priority
        self.description = description

    def __lt__(self, other):
        """Compare requests by priority"""
        return self.priority.value < other.priority.value

    def __repr__(self):
        return f"Emergency({self.name}, {self.priority.name})"

class MedicalEmergencyHandler:
    """
    Manages medical emergencies using Priority Queue

    Justification: Priority Queue is ideal for medical emergencies because:
    - Handles requests based on severity level, not arrival order
    - Ensures critical cases are treated immediately
    - Efficiently organizes multiple concurrent emergencies
    """
```

```python
    """

    def __init__(self):
        """Initialize emergency queue using deque with manual priority sorting"""
        self.emergencies = []

    def add_emergency(self, request):
        """
        Add new emergency to the system

        Args:
            request: EmergencyRequest object
        """
        self.emergencies.append(request)
        self.emergencies.sort(key=lambda x: x.priority.value)

    def get_next_emergency(self):
        """
        Retrieve the highest priority emergency

        Returns:
            EmergencyRequest with highest priority, or None if empty
        """
        if self.emergencies:
            return self.emergencies.pop(0)
        return None

    def display_queue(self):
        """Display all pending emergencies in priority order"""
        if not self.emergencies:
            print("No pending emergencies")
        for request in self.emergencies:
            print(f"  {request} - {request.description}")

```

```python

# Example usage
if __name__ == "__main__":
    handler = MedicalEmergencyHandler()

    # Add sample emergencies
    handler.add_emergency(EmergencyRequest("S001", "Raj Kumar", PriorityLevel.MEDIUM, "High fever"))
    handler.add_emergency(EmergencyRequest("S002", "Priya Singh", PriorityLevel.CRITICAL, "Severe allergic reaction"))
    handler.add_emergency(EmergencyRequest("S003", "Amit Patel", PriorityLevel.HIGH, "Chest pain"))

    print("Pending Emergencies:")
    handler.display_queue()

    print("\nProcessing emergencies in priority order:")
    while True:
        emergency = handler.get_next_emergency()
        if not emergency:
            break
        print(f"Attending to: {emergency}")
```

Output:-

```
C:\Users\ankam\OneDrive\Desktop\AIAC>C:/Users/ankam/AppData/Local/Programs/Python/Python312/python.exe
Pending Emergencies:
  Emergency(Priya Singh, CRITICAL) - Severe allergic reaction
  Emergency(Amit Patel, HIGH) - Chest pain
  Emergency(Raj Kumar, MEDIUM) - High fever

Processing emergencies in priority order:
Attending to: Emergency(Priya Singh, CRITICAL)
Attending to: Emergency(Amit Patel, HIGH)
Attending to: Emergency(Raj Kumar, MEDIUM)
```

Task #16: Smart Electricity Distribution System

Scenario:

An electricity board plans to develop a system that includes:

1. Complaint Handling
2. Fault Priority Handling
3. Consumer Record Lookup
4. Power Grid Connectivity
5. Meter Reading History Tracking

Student Task

• For each feature, select the most appropriate data structure from the list below:

o Stack
o Queue
o Priority Queue
o Linked List
o Binary Search Tree (BST)
o Graph
o Hash Table
o Deque

• Justify your choice in 2–3 sentences per feature.
• Implement one selected feature as a working Python program with AI-assisted code generation.

Expected Output:

• A table mapping feature → chosen data structure → justification.
• A functional Python program implementing the chosen feature with comments and docstrings.

“”“

Smart Electricity Distribution System – Data Structure Selection

An electricity board system must manage:

1. Complaint Handling

2. Fault Priority Handling

3. Consumer Record Lookup

4. Power Grid Connectivity

5. Meter Reading History Tracking

Tasks:

1. Choose the most appropriate data structure from the list:

   Stack, Queue, Priority Queue, Linked List, Binary Search Tree (BST), Graph, Hash Table, Deque.

2. Write 2–3 sentences justifying your choice for each feature.

3. Implement one selected feature as a Python program with comments and docstrings.

“”“”

Feature	**Data Structure**	**Justification**
Complaint Handling	Queue	Complaints should be processed in the order they are received. A Queue follows the FIFO (First-In-First-Out) principle, ensuring fairness. This makes complaint processing systematic.
Fault Priority Handling	Priority Queue	Some faults (like transformer failure) are more critical than others. A Priority Queue processes higher priority faults first. This helps restore power faster in critical areas.
Consumer Record Lookup	Hash Table	Consumer records can be stored using consumer ID as the key. Hash Tables allow very fast lookup in constant time. This makes retrieving consumer information efficient.
Power Grid Connectivity	Graph	Power stations, transformers, and areas can be represented as nodes. Connections between them are edges. Graphs help model and analyze the electricity distribution network.
Meter Reading History Tracking	Linked List	Meter readings are recorded over time. A Linked List allows easy insertion of new readings in sequence. It is efficient for maintaining historical records.

```python
import heapq

class FaultPrioritySystem:
    """Priority Queue system for handling electricity faults."""

    def __init__(self):
        """Initialize an empty fault priority queue."""
        self.fault_queue = []

    def report_fault(self, priority, description):
        """
        Add a fault to the priority queue.

        priority (int): Lower number means higher priority
        description (str): Description of the fault
        """
        heapq.heappush(self.fault_queue, (priority, description))
        print("Fault reported:", description)

    def resolve_fault(self):
        """
        Resolve the highest priority fault.
        """
        if self.fault_queue:
            priority, fault = heapq.heappop(self.fault_queue)
            print("Resolving fault:", fault)
        else:
            print("No faults pending")

# Example usage
system = FaultPrioritySystem()

system.report_fault(2, "Street light failure")
system.report_fault(1, "Transformer breakdown")
system.report_fault(3, "Minor voltage fluctuation")

system.resolve_fault()
system.resolve_fault()
```

Output:-

```
C:\Users\ankam\OneDrive\Desktop\AIAC>C:/Users/a
Fault reported: Street light failure
Fault reported: Transformer breakdown
Fault reported: Minor voltage fluctuation
Resolving fault: Transformer breakdown
Resolving fault: Street light failure
```

Task #17: Smart Logistics & Warehouse Management
Scenario:
A logistics company wants to implement:
1. Shipment Processing
2. Urgent Shipment Priority
3. Product Lookup by Code
4. Warehouse Network Connectivity
5. Inventory Add/Remove Operations
Student Task
• For each feature, select the most appropriate data structure from the list below:
o Stack
o Queue
o Priority Queue
o Linked List
o Binary Search Tree (BST)
o Graph
o Hash Table
o Deque
• Justify your choice in 2–3 sentences per feature.
• Implement one selected feature as a working Python program with AI-assisted code generation.
Expected Output:
• A table mapping feature → chosen data structure → justification.
• A functional Python program implementing the chosen feature with comments and docstrings.

```
"""

Smart Logistics & Warehouse Management – Data Structure Selection

A logistics company system must manage:

1. Shipment Processing

2. Urgent Shipment Priority

3. Product Lookup by Code

4. Warehouse Network Connectivity

5. Inventory Add/Remove Operations

Tasks:

1. Choose the most appropriate data structure from the list:

   Stack, Queue, Priority Queue, Linked List, Binary Search Tree (BST), Graph, Hash Table, Deque.

2. Write 2–3 sentences justifying your choice for each feature.

3. Implement one selected feature as a Python program with comments and docstrings.

"""
```

```python
from collections import deque

class ShipmentQueue:
    """Queue system for processing warehouse shipments."""

    def __init__(self):
        """Initialize an empty shipment queue."""
        self.queue = deque()

    def add_shipment(self, shipment_id):
        """
        Add a shipment to the queue.

        Args:
            shipment_id (str): Unique shipment identifier
        """
        self.queue.append(shipment_id)
        print("Shipment added:", shipment_id)

    def process_shipment(self):
        """
        Process the next shipment in the queue.
        """
        if self.queue:
            shipment = self.queue.popleft()
            print("Processing shipment:", shipment)
        else:
            print("No shipments to process")

# Example usage
warehouse = ShipmentQueue()

warehouse.add_shipment("SHP101")
warehouse.add_shipment("SHP102")
warehouse.add_shipment("SHP103")

warehouse.process_shipment()
warehouse.process_shipment()
```

Output:-

```
C:\Users\ankam\OneDrive\Desktop\AIAC>C:/Users/
Shipment added: SHP101
Shipment added: SHP102
Shipment added: SHP103
Processing shipment: SHP101
Processing shipment: SHP102
```

Task 18 (Collatz Sequence Generator – Test Case Design)

• Function: Generate Collatz sequence until reaching 1.
• Test Cases to Design:
• Normal: 6 → [6,3,10,5,16,8,4,2,1]
• Edge: 1 → [1]
• Negative: -5
• Large: 27 (well-known long sequence)
• Requirement: Validate correctness with pytest.

Explanation:

We need to write a function that:

• Takes an integer n as input.
• Generates the Collatz sequence (also called the 3n+1 sequence).
• The rules are:
o If n is even → next = n / 2.
o If n is odd → next = 3n + 1.
• Repeat until we reach 1.
• Return the full sequence as a list.

Example

Input: 6

Steps:

• 6 (even → 6/2 = 3)
• 3 (odd → 3*3+1 = 10)
• 10 (even → 10/2 = 5)
• 5 (odd → 3*5+1 = 16)
• 16 (even → 16/2 = 8)
• 8 (even → 8/2 = 4)
• 4 (even → 4/2 = 2)
• 2 (even → 2/2 = 1)

Output:

[6, 3, 10, 5, 16, 8, 4, 2, 1]

```python
"""
Collatz Sequence Generator with Test Cases

Write a Python function collatz_sequence(n) that generates the Collatz
(3n + 1) sequence until the number reaches 1.

Rules:
- If n is even → next number = n / 2
- If n is odd → next number = 3*n + 1
- Continue until the sequence reaches 1
- Return the full sequence as a list

Example:
Input: 6
Output: [6, 3, 10, 5, 16, 8, 4, 2, 1]
Requirements:
- Implement the function collatz_sequence(n)
- Handle invalid inputs such as negative numbers
- Write pytest test cases for:
    Normal case: 6 → [6,3,10,5,16,8,4,2,1]
    Edge case: 1 → [1]
    Negative case: -5 → should raise an error
    Large case: 27 → long sequence
- Use pytest to validate correctness.
"""
def collatz_sequence(n):
    """Generate the Collatz sequence for a given number n.
    Parameters:
    n (int): The starting number for the Collatz sequence. Must be a positive integer.
    Returns:
    list: A list containing the Collatz sequence starting from n and ending at 1.
    Raises:
    ValueError: If n is not a positive integer.
    """
    if n <= 0:
        raise ValueError("Input must be a positive integer.")

```

```python
    if n <= 0:
        raise ValueError("Input must be a positive integer.")

    sequence = []
    while n != 1:
        sequence.append(n)
        if n % 2 == 0:
            n = n // 2
        else:
            n = 3 * n + 1
    sequence.append(1)  # Append the final element of the sequence
    return sequence
# Example usage
if __name__ == "__main__":
    try:
        print(collatz_sequence(6))  # Normal case
        print(collatz_sequence(1))  # Edge case
        print(collatz_sequence(-5)) # Negative case (should raise an error)
    except ValueError as e:
        print(e)
    print(collatz_sequence(27))  # Large case
```

Output:-

```
[6, 3, 10, 5, 16, 8, 4, 2, 1]
[1]
Input must be a positive integer.
[27, 82, 41, 124, 62, 31, 94, 47, 142, 71, 214, 107, 322, 161, 484, 242, 121, 364, 182, 91, 274, 137, 412, 206, 103, 310, 155, 466, 233, 700, 350, 175
, 526, 263, 790, 395, 1186, 593, 1780, 890, 445, 1336, 668, 334, 167, 502, 251, 754, 377, 1132, 566, 283, 850, 425, 1276, 638, 319, 958, 479, 1438, 71
9, 2158, 1079, 3238, 1619, 4858, 2429, 7288, 3644, 1822, 911, 2734, 1367, 4102, 2051, 6154, 3077, 9232, 4616, 2308, 1154, 577, 1732, 866, 433, 1300, 6
50, 325, 976, 488, 244, 122, 61, 184, 92, 46, 23, 70, 35, 106, 53, 160, 80, 40, 20, 10, 5, 16, 8, 4, 2, 1]
```

Task 19 (Lucas Number Sequence – Test Case Design)

• Function: Generate Lucas sequence up to n terms.
(Starts with 2,1, then Fn = Fn-1 + Fn-2)
• Test Cases to Design:
• Normal: 5 → [2, 1, 3, 4, 7]
• Edge: 1 → [2]
• Negative: -5 → Error
• Large: 10 (last element = 76).
• Requirement: Validate correctness with pytest.

```python
"""
Lucas Number Sequence Generator with Test Cases

Write a Python function lucas_sequence(n) that generates the first n
terms of the Lucas sequence.

Lucas sequence rules:
- First two numbers are 2 and 1
- Each next number = previous two numbers added
  Fn = Fn-1 + Fn-2

Example:
Input: 5
Output: [2, 1, 3, 4, 7]

Requirements:
- Implement lucas_sequence(n) that returns a list
- Handle invalid input such as negative numbers
- Write pytest test cases for:
    Normal case: 5 → [2,1,3,4,7]
    Edge case: 1 → [2]
    Negative case: -5 → should raise an error
    Large case: 10 → last element should be 76
- Use pytest to validate correctness.
"""
def lucas_sequence(n):
    """Generate the first n terms of the Lucas sequence."""

    if n < 0:
        raise ValueError("Input must be a non-negative integer.")

    if n == 0:
        return []
    elif n == 1:
        return [2]
    elif n == 2:
        return [2, 1]

    sequence = [2, 1]

    for i in range(2, n):
        next_term = sequence[i-1] + sequence[i-2]
        sequence.append(next_term)

    return sequence

```

```python

# Example usage
if __name__ == "__main__":
    try:
        print(lucas_sequence(5))   # Normal case
        print(lucas_sequence(1))   # Edge case
        print(lucas_sequence(-5))  # Negative case
    except ValueError as e:
        print(e)

    print(lucas_sequence(10))     # Large case
```

Output:-

```
C:\Users\ankam\OneDrive\Desktop\AIAC>C:/Users/ankam/AppData/Local/Programs/
[2, 1, 3, 4, 7]
[2, 1, 3, 4, 7]
[2]
Input must be a non-negative integer.
[2, 1, 3, 4, 7, 11, 18, 29, 47, 76]

C:\Users\ankam\OneDrive\Desktop\AIAC>
```

Task 20 (Vowel & Consonant Counter – Test Case Design)

• Function: Count vowels and consonants in string.

• Test Cases to Design:

• Normal: "hello" → (2,3)

• Edge: "" → (0,0)

• Only vowels: "aeiou" → (5,0)

Large: Long text

• Requirement: Validate correctness with pytest.

```python
"""
Vowel and Consonant Counter with Test Cases

Write a Python function count_vowels_consonants(text) that counts
the number of vowels and consonants in a given string.

Rules:
- Vowels: a, e, i, o, u
- All other alphabet characters are consonants
- Ignore spaces and special characters if needed

Example:
Input: "hello"
Output: (2,3)

Requirements:
- Implement the function count_vowels_consonants(text)
- Return a tuple (vowel_count, consonant_count)
- Write pytest test cases for:
    Normal case: "hello" → (2,3)
    Edge case: "" → (0,0)
    Only vowels: "aeiou" → (5,0)
    Large case: long paragraph text
- Use pytest to validate correctness.
"""
def count_vowels_consonants(text):
    """Count the number of vowels and consonants in a given string.
    Parameters:
    text (str): The input string to analyze.
    Returns:
    tuple: A tuple containing the count of vowels and consonants (vowel_count, consonant_count).
    """
    vowels = 'aeiouAEIOU'
    vowel_count = 0
    consonant_count = 0
```

```python
    consonant_count = 0

    for char in text:
        if char.isalpha():  # Check if the character is an alphabet
            if char in vowels:
                vowel_count += 1
            else:
                consonant_count += 1

    return (vowel_count, consonant_count)
# Example usage
if __name__ == "__main__":
    print(count_vowels_consonants("hello"))  # Normal case
    print(count_vowels_consonants(""))       # Edge case
    print(count_vowels_consonants("aeiou"))  # Only vowels case
    # Large case can be tested with a long paragraph of text
    print(count_vowels_consonants("This is a large paragraph with many words."))
```

Output:-

```
C:\Users\ankam\OneDrive\Desktop\AIAC>C:/Users/ankam/AppData
(2, 3)
(0, 0)
(5, 0)
(11, 23)
```